

EG2300 — Engineering Design

Loadable Instruction ROM (16×16) + PC Addressed Read

1 Overview

In this workshop you will build the **instruction memory** for a very simple CPU. The instruction memory stores **16 instructions** (addresses 0–15), each **16 bits** wide. A single push button loads a complete programme into memory, and a 4-bit **Program Counter (PC)** selects which instruction is read out.

Target behaviour

- **Instruction width:** 16 bits
- **Memory depth:** 16 words (rows)
- **Load operation:** On the **rising edge** of LOAD_BTN, all 16 input words are captured simultaneously.
- **Read operation:** ROM_OUT = ROM[PC] (combinational read)
- **Binary only:** Use binary for inputs/outputs and checks (no hexadecimal).

2 Learning objectives

By the end of this workshop you should be able to:

1. Build a 16-bit **parallel-in parallel-out** register using library D flip-flops.
2. Use **hierarchy** (subcircuits) to create clean, reusable blocks.
3. Construct a structured **16×16** loadable ROM from repeated register blocks.
4. Implement a **PC-addressed selection** to output the correct stored word.

3 What to save

Your file must include the following subcircuits (exact names recommended):

- Reg16_PIP0
- ROM16x16_Loadable
- ROM_ReadSelect
- Top_W1_Test

4 Allowed components

You may use Logisim library components for:

- D flip-flops
- Multiplexers, splitters, pins, probes, constants

You must still build the ROM **structurally** from repeated register blocks.

5 Interface specification

All widths must match the table below.

Signals

Name	Width	Dir.	Meaning
IN0 ... IN15	16	in	Sixteen 16-bit instruction words to be loaded into ROM rows 0–15
LOAD_BTN	1	in	Push button used to clock the load (capture on rising edge)
P.C.	4	in	Program Counter address (4-bits, 0–15)
ROM_OUT	16	out	Selected instruction output

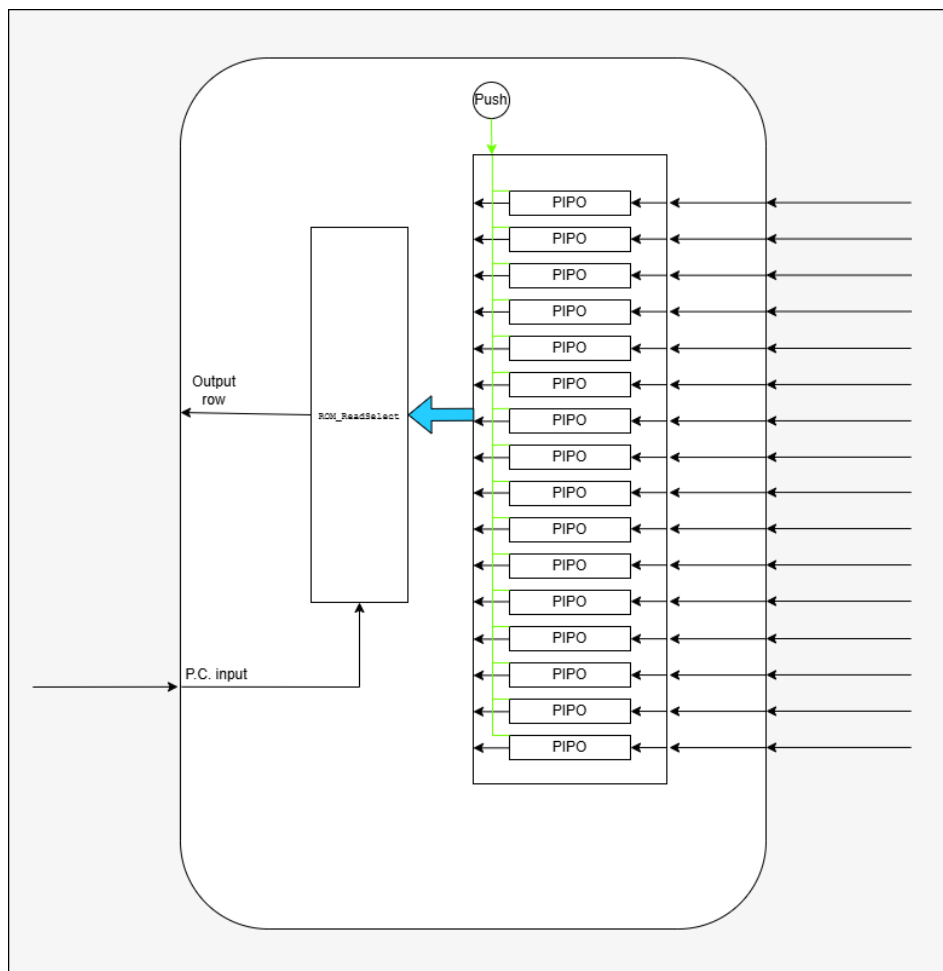


Figure 1: Block diagram of the loadable instruction ROM and PC read-select block.

6 Tasks

6.1 Part A — Build a 16-bit register (Reg16_PIP0)

Goal: Create a 16-bit parallel-in parallel-out register using library D flip-flops.

A1. Subcircuit: Reg16_PIP0

Pins

- Inputs: D[15:0], CLK
- Output: Q[15:0]

Build instructions

1. Place 16 D flip-flops.
2. Wire each D[i] to the corresponding flip-flop data input.
3. Wire all flip-flop clock pins to CLK.
4. Combine the 16 Q outputs into Q[15:0] (use splitters/buses neatly).

Required self-test

- Drive D with a known 16-bit binary pattern.
- Toggle CLK once: Q must update only on the rising edge and hold its value otherwise.

6.2 Part B — Build the 16×16 loadable ROM (ROM16x16_Loadable)

Goal: Create 16 stored words (rows), each 16 bits wide, that all load together on a single button press.

B1. Subcircuit: ROM16x16_Loadable

Pins

- Inputs: IN0[15:0] ... IN15[15:0], LOAD_BTN
- Outputs: ROW0[15:0] ... ROW15[15:0] (for visibility and marking)

Build instructions

1. Place 16 copies of Reg16_PIP0, one per row.
2. Connect IN0 to row 0 register input, IN1 to row 1, ..., IN15 to row 15.
3. Connect LOAD_BTN to the CLK input of **all** 16 registers.
4. Expose each register output as ROW0 ... ROW15.

Required behaviour

- One rising edge on LOAD_BTN must capture all 16 input words simultaneously.
- Stored values must remain stable after loading (until the next rising edge load).

B2. ROM load test (required)

Use this suggested debug pattern (binary only):

- IN0 = 0000 0000 0000 0000
- IN1 = 0001 0001 0001 0001
- IN2 = 0010 0010 0010 0010
- ...
- IN15 = 1111 1111 1111 1111

Press LOAD_BTN once and verify ROW0 .. ROW15 match the intended values.

6.3 Part C — Build the PC read/select block (ROM_ReadSelect)

Goal: Output the ROM row indexed by the 4-bit Program Counter (P.C.).

C1. Subcircuit: ROM_ReadSelect

Pins

- Inputs: PC[3:0], ROW0[15:0] ... ROW15[15:0]
- Output: ROM_OUT[15:0]

Build instructions (bit-slice implementation — required)

1. Use splitters to break each row bus into individual bits:

$$\text{ROW}_k[15:0] \rightarrow \text{ROW}_k[15], \dots, \text{ROW}_k[0]$$

for each row $k = 0, 1, \dots, 15$.

2. For each bit position $b \in \{0, \dots, 15\}$, build a 1-bit-wide 16-to-1 selection:

- Place a **1-bit** 16-to-1 multiplexer.
- Connect ROW0[b] ... ROW15[b] to the mux data inputs (input 0 is ROW0[b], input 15 is ROW15[b]).
- Connect PC[3:0] to the mux select input.
- The mux output is ROM_OUT[b].

3. Repeat Step 2 for all 16 bits ($b = 0$ to 15), giving **16 multiplexers in parallel**.

4. Combine ROM_OUT[15] ... ROM_OUT[0] back into the 16-bit bus ROM_OUT[15:0] using a splitter.

Required test After loading the ROM, step P.C. through values 0–15 and verify:

$$\text{PC} = 0000 \Rightarrow \text{ROM_OUT} = \text{ROW}_0, \quad \dots, \quad \text{PC} = 1111 \Rightarrow \text{ROM_OUT} = \text{ROW}_{15}.$$

6.4 Part D — Integration (Top_W1_Test)

Goal: One top-level circuit that loads a programme and reads it by PC.

D1. Subcircuit: Top_W1_Test

Include

- 16-bit sources for IN0 ... IN15 (constants or input pins)
- A push button for LOAD_BTN
- A 4-bit input for PC
- Output probe/pin for ROM_OUT set to **binary**

Final pass condition

- After one press of LOAD_BTN, the stored values remain fixed.
- Changing PC always selects the correct stored row.

7 Optional extension (if you finish early)

1. Add an **Instruction Register (IR)** that latches ROM_OUT on a separate step clock.
2. Build a 4-bit step counter to drive PC and demonstrate automatic stepping through instructions.
3. Include reset push button to clear the ROM.

Reminder: Your design must be modular and readable. Use subcircuits and buses.